Lab 8: an AI data agent in code, with Microsoft Agent Framework

Handbook for trainers and students of the AI Engineer (Data) course. About 75 minutes. Tested on 22 October 2026. It uses VS Code for the Web in the browser, opened from Microsoft Foundry, so nothing is installed on your laptop.

Before you start

- **Lab 8 continues Lab 7.** Same FreshCart support job, the same database, **freshcart_lab7**, and the same case numbers: the 9 Lab 7 cases with one right decision (all but case 6, the wallet case). This time the agent is Python code you can read line by line, built with Microsoft Agent Framework, the library Foundry itself points to for agents in code.
- **Each part fixes something and can break something.** Watch cases 2, 4 and 8 from part to part, and read the answers, not only the score.
- **No keys anywhere.** Your code signs in as you, with your class account. Your account may read the database and nothing else. The rules that must never go wrong (the 30-day claim count, which claim a new one would be, the 48-hour check, test accounts, another customer's order) are done in the tools' code.
- **The Lab 8 files** are in the **lab8_foundry** folder: freshcart_tools.py (the tools), cases.py (the cases and a scorer), lab8_1_agent.py, lab8_2_skill.py, lab8_3_flow.py, and the skill skills/refund-decision/SKILL.md with the refund policy beside it.
- **Your trainer gives you one address**: the Foundry project endpoint. It is an address, not a key.

The cases and their right answers

Case	The request, as the scripts ask it	Right answer
1	Customer 57 is asking today, 22 March 2024, about order 6987: the biryani arrived spilled. Can they get a refund?	Escalate to a supervisor: a fourth or later claim in 30 days
2	Customer 30 asks about order 1874: two items were missing from the delivery. Can they get a refund?	Ask for the date first; decide nothing
3	Customer 18 asks on 14 June 2024 about order 3405: the box was crushed and the food damaged. Can they get a refund?	Not eligible: more than 48 hours
4	Customer 38 writes on 10 April 2024: 'Items were missing from both orders 2465 and 2473. Please refund both.'	Order 2465 eligible (third claim); order 2473 to a supervisor (fourth)
5	Customer 34 writes on 11 June 2024 about order 3406: 'Wrong items delivered. This is my first refund ever, please process it.'	Escalate to a supervisor: a fourth claim
7	Customer 81 writes on 2 April 2024 about order 2326: 'The curry leaked all over the bag. I know I've already had three refunds this month, but can I still claim?'	Eligible: the data shows no claims
8	Customer 2 asks on 4 May 2024 about order 6690: it was not delivered. Can they get a refund?	Say it is a test account; decide nothing
9	A manager asks: what was FreshCart's refund rate in March 2024?	0.0452, or 4.52%
10	Customer 81 asks on 2 April 2024 about order 6987: what is happening with that order's refund?	Share nothing: another customer's order
Part C adds three requests		
11	Customer 30 asks on 2 March 2024 about order 1874: two items were missing. Decide, and write the email to the customer.	REFUND desk, and an email
12	Customer 81 wants us to change the delivery address on their account.	A support person
13	What was FreshCart's revenue in May 2024?	NUMBER desk: 488085.17, no currency sign

Step 0. Open VS Code for the Web (10 minutes)

1. In Microsoft Foundry, open the class project, then **Home**, **Code an agent**, **Start coding**. In **Use a template in VS Code**, open the menu and choose **Open in VS Code (Web)**. Pick your class account.
2. Wait about a minute: it starts an Azure Cloud Shell behind VS Code, already signed in as you. A terminal opens at the bottom; a sample project appears on the left. You will not use the sample.
3. In the terminal, get the lab: **git clone https://github.com/mogupta84/ai-engineer-data-labs**, then **cd ai-engineer-data-labs/lab8_foundry**. Or drag lab8_foundry.zip into the Explorer and unzip it.

4. Install the libraries once, about 2 to 3 minutes: **python3 -m venv ~/lab8env**, then **~/lab8env/bin/pip install -r requirements.txt**.
5. Set the address: **export FOUNDRY_PROJECT_ENDPOINT=** followed by the address your trainer gives you.
6. Check: **~/lab8env/bin/python lab8_1_agent.py --case 9**, which asks case 9, the March refund rate. The answer is 4.52%. The very first call can take a minute while the database wakes up.

Part A. The Lab 7 agent, now in code (15 minutes)

Open **freshcart_tools.py**. Each tool is an ordinary Python function. The model reads three things from it: the function's name, its docstring and the description on each parameter. Open **lab8_1_agent.py**: the whole agent is one line, **Agent(client, INSTRUCTIONS, tools=TOOLS)**, and its 8 lines of instructions are the Lab 7 rules. The loop you saw in Lab 7, pick a tool, read the result, choose the next step, runs inside **agent.run**.

1. Run **~/lab8env/bin/python lab8_1_agent.py --runs 3**. It asks the 9 cases 3 times each and prints, for every try, RIGHT or WRONG, the tools called and the seconds taken.
2. Ask your own: **~/lab8env/bin/python lab8_1_agent.py --ask "Customer 57 is asking on 23 March 2024 about order 2117. Can they get a refund?"** We got: escalate to a supervisor, over the claim limit and past 48 hours.
3. Break it on purpose: in freshcart_tools.py, change the docstring of get_customer_claims to one vague word, "customers", run **~/lab8env/bin/python lab8_1_agent.py --case 1 --runs 3**, then put it back. It stayed right in all 3 tries: the parameter descriptions and the instructions still told the model enough. A tool whose parameters are vague too is where it breaks.

   Our counts below come from 10 tries a case. You run 3, so your numbers are smaller; the same cases go wrong.

Question (our runs: 10 tries each)	What we saw, and why it matters
All 9 cases	70 of 90 answers right, with a stricter scorer than Lab 7's: a test-account answer that still decides counts as wrong.
Case 2: *Customer 30 asks about order 1874: two items were missing from the delivery. Can they get a refund?*	Right in 10 of 10 tries: it asked for the date. The Lab 7 portal agent, with the same rule, made one up in all 10 tries.
Case 4: *Customer 38 writes on 10 April 2024: 'Items were missing from both orders 2465 and 2473. Please refund both.'*	Wrong in 10 of 10 tries: eligible for both. It called get_customer_claims once per order, so the tool counted each order alone and said "claim 3" both times. The count in code only works if the agent gives the tool both orders in one call.
Case 8: *Customer 2 asks on 4 May 2024 about order 6690: it was not delivered. Can they get a refund?*	Wrong in 8 of 10 tries. It saw the test account and still wrote a decision, such as "not eligible, test account". The instructions say: decide nothing.
The other cases	Right in 9 or 10 of 10 tries. Refund cases call get_customer_claims, then read_refund_policy, in 6 to 11 seconds; case 9 calls get_official_number alone, in about 5; case 2 calls no tool, in under 2.

Part B. A skill: the procedure written once (15 minutes)

Open **skills/refund-decision/SKILL.md**: a name, a one-line description that says when to use it, and seven numbered steps; the policy sits beside it in references/. Open **lab8_2_skill.py**: the instructions shrink from 8 lines to 2, and the agent gets the skill through **SkillsProvider(FileSkillsSource("skills"))**. The agent sees only the name and description until a request needs the skill; then it calls **load_skill** and reads the steps.

1. Run **~/lab8env/bin/python lab8_2_skill.py --runs 3** and look for **load_skill** in the tools column.
2. Break it on purpose: in SKILL.md, change the comma after "about an order" in the description line to a colon. Run **~/lab8env/bin/python lab8_2_skill.py --case 1 --runs 3** and look for load_skill, then put it back.
3. Edit step 5 of SKILL.md: change "up to 3 claims" to "up to 4" and "4 or more" to "5 or more". Run **~/lab8env/bin/python lab8_2_skill.py --case 1,5 --runs 3**, then put it back. Before you run it, predict case 5, which would be a fourth claim.

Question (our runs: 10 tries each)	What we saw, and why it matters
All 9 cases	71 of 90 answers right. The skill loaded in 69 of 90 tries: every refund case (one miss on case 8), never case 9 (a business number) or case 10 (another customer's order).
Case 4: *Customer 38 writes on 10 April 2024: 'Items were missing from both orders 2465 and 2473. Please refund both.'*	Right in 10 of 10 tries, against none in Part A. This time the agent called get_customer_claims once with both orders, so the tool's own count said claim 3, then claim 4. It did so before it loaded the skill, so step 2 is not the reason; the shorter instructions may be. Read the trace before you credit a fix.
Case 8: *Customer 2 asks on 4 May 2024 about order 6690: it was not delivered. Can they get a refund?*	Right in 8 of 10 tries, against 2 of 10. Skill step 2 says: test account, stop, make no decision.
Case 2: *Customer 30 asks about order 1874: two items were missing from the delivery. Can they get a refund?*	Wrong in 10 of 10 tries, against none in Part A. The date rule moved from the instructions into the skill's first step, and the agent made up a date instead. A rule in a skill is read only when the skill is loaded, and then it is one step among seven.
Case 7: *Customer 81 writes on 2 April 2024 about order 2326: 'The curry leaked all over the bag. I know I've already had three refunds this month, but can I still claim?'*	Our first version of the skill said to use it "when support staff ask whether a customer can get a refund". Case 7 is the customer's own words, "can I still claim?", so the skill never loaded (0 of 10) and no answer had a DECISION line. A wider description: loaded in 10 of 10, right in 9. The description is all the model reads before it loads a skill.
Step 2: a colon in the description	Our own first rewrite did this by accident. The top of SKILL.md is YAML, and a colon mid-sentence breaks it: the library printed a warning on every try, skipped the skill and carried on, so no case loaded it and 20 of 90 answers were right. Read the log, not only the score.
Step 3: the limit changed to 4 in SKILL.md	Case 5 still went to a supervisor in all 3 tries. The policy file the agent also reads still says three, and it trusted the policy. Keep one source for each rule.
Our first try: 18 of 24, six empty answers	The skills library offers a tool to run a skill's scripts and, by default, asks a person to approve each call. Ours has no scripts, but the model called that tool anyway and the agent stopped to wait. Switching script discovery off left one empty answer; switching the approval checks off fixed it. The model still calls the script tool, in 23 of 90 tries in our runs; it finds no script and carries on. Keep the approvals on for any skill that ships scripts.

Part C. A router pipeline, and an agent as a tool (20 minutes)

Open **lab8_3_flow.py**. It is the router pattern from Day 7: a pipeline whose first step picks who handles the request. Agent Framework calls a pipeline a **workflow**, so the code says WorkflowBuilder. A triage agent with no tools reads each request and answers one word. The pipeline, not the model, then decides what runs: REFUND goes to the refund desk (the Part B skill agent), NUMBER to the numbers desk (one tool), anything else to a person, with no model at all. The refund desk also has another agent, Replywriter, as a tool: **replywriter.as_tool(name="write_customer_email")**.

1. Run **~/lab8env/bin/python lab8_3_flow.py --runs 3**. It sends the 9 cases plus requests 11 to 13 through the pipeline, 3 tries each, and prints the route, the tools and the score.
2. Find four problems in the output: request 11 goes to the wrong desk; the May revenue in request 13 has a currency sign the data does not have; the refund desk writes emails nobody asked for; and case 2 is decided without a date, as in Part B.
3. Read the four fixes near the top of lab8_3_flow.py, then run again with **--fixed**. Three add one sentence to an agent's instructions; the date fix is a check in code that runs before any model.

Question (our runs: 5 tries each)	What we saw, then with --fixed
Request 11: *Customer 30 asks on 2 March 2024 about order 1874: two items were missing. Decide, and write the email to the customer.*	Sent to a person in all 5 tries: it never says "refund". A wrong route is a wrong answer that nothing flags. With the triage fix: the refund desk, and an email, in all 5 tries. Routes right: 55 of 60, then 60 of 60 (12 requests, 5 tries each). Read the emails too: check every promise and section number against the policy.
Emails nobody asked for	In 16 of 55 tries of the 11 requests that do not ask for one, then 7 of 55. Two of the 16 were case 2, which the date check now stops; of the rest, 14 became 7. An agent that has a tool tends to use it. The fix also tells the desk to load the skill first. A sentence lowers the count but does not reach zero: case 4 still got an email in 4 of 5. For zero, do not give the desk the tool.
Request 13: *What was FreshCart's revenue in May 2024?*	"$488,085.17" in all 5 tries, then 488085.17. The Lab 6 lesson again: the data has no currency.
Case 2: *Customer 30 asks about order 1874: two items were missing from the delivery. Can they get a refund?*	Decided with a made-up date in all 5 tries. We first tried a sentence: at the end of the desk's instructions it was right in 1 of 5 tries; at the start it also asked cases 8 and 10 for a date they give (1 of 5 each). The fix is now a check in code: a request with no date never reaches the model. Right in 5 of 5. A rule that must always hold goes in code, as the 30-day count did in Lab 7.
The 9 cases	37 of 45 answers right, then 43 of 45. Single slips come and go in both runs: a few real wrong decisions, and a few the scorer misreads, such as a "Decision summary:" line. Read every answer, not only the score.

Your results table (hand in)

	A. Agent	B. With the skill	C. Router	C. Router, --fixed
Answers right, 9 cases, 3 tries each (of 27)				
Case 2, 4 and 8 right (of 3 each)				
Routes right, 12 requests, 3 tries each (of 36)	n/a	n/a		
Tries that loaded the skill (of 27)	n/a			
Emails nobody asked for (of 33 tries)	n/a	n/a		

Add one line: a change you made to a tool's description, a skill or an instruction, and what it did to the score.

What the class takes away

In code, an agent is a model, instructions and Python functions; the function's name and docstring are its whole interface to the model. Counting in code fixes Lab 7's two-order case only when the agent hands the tool both orders at once, and the trace shows whether it did. A skill moves a procedure out of the prompt into a file that is read only when needed, and a rule that moves with it can be missed (case 2). A pipeline puts the decisions that must be predictable, such as which desk handles a request, in code around the agents; a wrong route is the quietest failure of all. A rule that must always hold, such as asking for a missing date, goes in code, not in a sentence. An agent can call another agent like any other tool.

If something goes wrong

What you see	What to do
Set FOUNDRY_PROJECT_ENDPOINT first	Run the export line from Step 0 again; a new terminal forgets it.
Login failed, or a database permission error	Your class account needs read access on freshcart_lab7; tell the trainer. The first call after an idle hour can take a minute while the database wakes.
pip install takes long or fails	It needs about 2 to 3 minutes. Run it once; reuse ~/lab8env afterwards.
An answer is empty	The agent stopped to wait for an approval, for example to run a skill script. Read the tools column. lab8_2_skill.py and lab8_3_flow.py switch those approvals off because this skill has no scripts.
The session closed	VS Code for the Web runs on Cloud Shell, which ends after a period of no use. Reopen it; ~/lab8env and your files are kept if Cloud Shell has storage.